

Computación y Estructuras Discretas III

Andrés A. Aristizábal P. aaaristizabal@icesi.edu.co
Angela Villota - apvillota@icesi.edu.co

Departamento de Computación y Sistemas Inteligentes



2 de febrero de 2026

- Power
- Kleene Closure
- Reverse
- Languages

1 Regular languages and expressions

- Regular Languages
- Regular Expressions

2 Regular expressions in Python

- Exercises in Python

Agenda

- Power
- Kleene Closure
- Reverse
- Languages

1 Regular languages and expressions

- Regular Languages
- Regular Expressions

2 Regular expressions in Python

- Exercises in Python

What is the power of a language?

Definition

Given a language A over Σ , ($A \subseteq \Sigma^*$), and a natural number $n \in \mathbb{N}$, A^n is defined in the following way:

$$\begin{aligned} A^0 &= \{\lambda\}, \\ A^n &= \underbrace{AA \cdots A}_{n \text{ times}} = \{u_1 \cdots u_n \mid u_i \in A, \text{ for all } i, 1 \leq i \leq n\}. \end{aligned}$$

A^2 is the set of double string concatenations of A , A^3 the set of triple string concatenations of A and in general A^n is the set of n string concatenations of A .

Agenda

- Power
- **Kleene Closure**
- Reverse
- Languages

1 **Regular languages and expressions**

- Regular Languages
- Regular Expressions

2 **Regular expressions in Python**

- Exercises in Python

Definition

The Kleene closure of A , $A \subseteq \Sigma^$, is the union of all powers of A and it is represented by A^* .*

$$\begin{aligned} A^* &= \text{set of all string concatenations of } A, \\ &\quad \text{including } \lambda \\ &= \{u_1 \cdots u_n \mid u_i \in A, n \geq 0\} \end{aligned}$$

Definition

The positive closure of a language A , $A \subseteq \Sigma^$, is the union of all powers of A , without including λ and is represented by A^+ .*

$$\begin{aligned} A^+ &= \text{set of all string concatenations of } A, \\ &\quad \text{without including } \lambda \\ &= \{u_1 \cdots u_n \mid u_i \in A, n \geq 1\} \end{aligned}$$

Which are the properties of $*$ and $+$?

Definition

- 1 $A^+ = A^* \cdot A = A \cdot A^*$.
- 2 $A^* \cdot A^* = A^*$.
- 3 $(A^*)^n = A^*$, for all $n, n \geq 1$.
- 4 $(A^*)^* = A^*$.
- 5 $A^+ \cdot A^+ \subseteq A^+$.
- 6 $(A^*)^+ = A^*$.
- 7 $(A^+)^* = A^*$.
- 8 $(A^+)^+ = A^+$.
- 9 If A and B are languages over Σ^* , then $(A \cup B)^* = (A^* B^*)^*$.

Agenda

- Power
- Kleene Closure
- **Reverse**
- Languages

1 Regular languages and expressions

- Regular Languages
- Regular Expressions

2 Regular expressions in Python

- Exercises in Python

Definition

Given A , a language over Σ , we define A^R in the following way:

$$A^R = \{u^R \mid u \in A\}$$

A^R is the inverse of A .

Agenda

- Power
- Kleene Closure
- Reverse
- Languages

1 Regular languages and expressions

- Regular Languages
- Regular Expressions

2 Regular expressions in Python

- Exercises in Python

What is the power of a language?

What is the power of a language?

Definition

Given a language A over Σ , ($A \subseteq \Sigma^$), and a natural number $n \in \mathbb{N}$, A^n is defined in the following way:*

What is the power of a language?

Definition

Given a language A over Σ , ($A \subseteq \Sigma^*$), and a natural number $n \in \mathbb{N}$, A^n is defined in the following way:

$$\begin{aligned} A^0 &= \{\lambda\}, \\ A^n &= \underbrace{AA \cdots A}_{n \text{ times}} = \{u_1 \cdots u_n \mid u_i \in A, \text{ for all } i, 1 \leq i \leq n\}. \end{aligned}$$

What is the power of a language?

Definition

Given a language A over Σ , ($A \subseteq \Sigma^*$), and a natural number $n \in \mathbb{N}$, A^n is defined in the following way:

$$\begin{aligned} A^0 &= \{\lambda\}, \\ A^n &= \underbrace{AA \cdots A}_{n \text{ times}} = \{u_1 \cdots u_n \mid u_i \in A, \text{ for all } i, 1 \leq i \leq n\}. \end{aligned}$$

A^2 is the set of double string concatenations of A , A^3 the set of triple string concatenations of A and in general A^n is the set of n string concatenations of A .

What is the Kleene closure of a language?

What is the Kleene closure of a language?

Definition

The Kleene closure of A , $A \subseteq \Sigma^$, is the union of all powers of A and it is represented by A^* .*

What is the Kleene closure of a language?

Definition

The Kleene closure of A , $A \subseteq \Sigma^$, is the union of all powers of A and it is represented by A^* .*

$$\begin{aligned} A^* &= \text{set of all string concatenations of } A, \\ &\quad \text{including } \lambda \\ &= \{u_1 \cdots u_n \mid u_i \in A, n \geq 0\} \end{aligned}$$

What is the positive closure of a language?

What is the positive closure of a language?

Definition

The positive closure of a language A , $A \subseteq \Sigma^$, is the union of all powers of A , without including λ and is represented by A^+ .*

What is the positive closure of a language?

Definition

The positive closure of a language A , $A \subseteq \Sigma^$, is the union of all powers of A , without including λ and is represented by A^+ .*

$$\begin{aligned} A^+ &= \text{set of all string concatenations of } A, \\ &\quad \text{without including } \lambda \\ &= \{u_1 \cdots u_n \mid u_i \in A, n \geq 1\} \end{aligned}$$

Which are the properties of $*$ and $+$?

Which are the properties of $*$ and $+$?

Definition

$$1 \quad A^+ = A^* \cdot A = A \cdot A^*.$$

Which are the properties of $*$ and $+$?

Definition

1 $A^+ = A^* \cdot A = A \cdot A^*.$

2 $A^* \cdot A^* = A^*.$

Which are the properties of $*$ and $+$?

Definition

- 1 $A^+ = A^* \cdot A = A \cdot A^*$.
- 2 $A^* \cdot A^* = A^*$.
- 3 $(A^*)^n = A^*$, *para todo* $n \geq 1$.

Which are the properties of $*$ and $+$?

Definition

- 1 $A^+ = A^* \cdot A = A \cdot A^*$.
- 2 $A^* \cdot A^* = A^*$.
- 3 $(A^*)^n = A^*$, *para todo* $n \geq 1$.
- 4 $(A^*)^* = A^*$.

Which are the properties of $*$ and $+$?

Definition

- 1 $A^+ = A^* \cdot A = A \cdot A^*$.
- 2 $A^* \cdot A^* = A^*$.
- 3 $(A^*)^n = A^*$, *para todo* $n \geq 1$.
- 4 $(A^*)^* = A^*$.
- 5 $A^+ \cdot A^+ \subseteq A^+$.

Which are the properties of $*$ and $+$?

Definition

- 1 $A^+ = A^* \cdot A = A \cdot A^*$.
- 2 $A^* \cdot A^* = A^*$.
- 3 $(A^*)^n = A^*$, *para todo* $n \geq 1$.
- 4 $(A^*)^* = A^*$.
- 5 $A^+ \cdot A^+ \subseteq A^+$.
- 6 $(A^*)^+ = A^*$.

Which are the properties of $*$ and $+$?

Definition

- 1 $A^+ = A^* \cdot A = A \cdot A^*$.
- 2 $A^* \cdot A^* = A^*$.
- 3 $(A^*)^n = A^*$, *para todo* $n \geq 1$.
- 4 $(A^*)^* = A^*$.
- 5 $A^+ \cdot A^+ \subseteq A^+$.
- 6 $(A^+)^+ = A^+$.
- 7 $(A^+)^* = A^*$.

Which are the properties of $*$ and $+$?

Definition

- 1 $A^+ = A^* \cdot A = A \cdot A^*$.
- 2 $A^* \cdot A^* = A^*$.
- 3 $(A^*)^n = A^*$, *para todo* $n \geq 1$.
- 4 $(A^*)^* = A^*$.
- 5 $A^+ \cdot A^+ \subseteq A^+$.
- 6 $(A^*)^+ = A^*$.
- 7 $(A^+)^* = A^*$.
- 8 $(A^+)^+ = A^+$.

Which are the properties of $*$ and $+$?

Definition

- 1 $A^+ = A^* \cdot A = A \cdot A^*$.
- 2 $A^* \cdot A^* = A^*$.
- 3 $(A^*)^n = A^*$, para todo $n \geq 1$.
- 4 $(A^*)^* = A^*$.
- 5 $A^+ \cdot A^+ \subseteq A^+$.
- 6 $(A^*)^+ = A^*$.
- 7 $(A^+)^* = A^*$.
- 8 $(A^+)^+ = A^+$.
- 9 If A and B are languages over Σ^* , then $(A \cup B)^* = (A^* B^*)^*$.

What is the inverse of a language?

What is the inverse of a language?

Definition

Given A , a language over Σ , we define A^R in the following way:

$$A^R = \{u^R \mid u \in A\}$$

What is the inverse of a language?

Definition

Given A , a language over Σ , we define A^R in the following way:

$$A^R = \{u^R \mid u \in A\}$$

A^R is the inverse of A .

Which are the properties of the inverse of a language?

Which are the properties of the inverse of a language?

Definition

Given A and B languages over Σ ($A, B \subseteq \Sigma^$).*

Which are the properties of the inverse of a language?

Definition

Given A and B languages over Σ ($A, B \subseteq \Sigma^*$).

$$\textcircled{1} \quad (A \cdot B)^R = B^R \cdot A^R.$$

Which are the properties of the inverse of a language?

Definition

Given A and B languages over Σ ($A, B \subseteq \Sigma^*$).

- 1 $(A \cdot B)^R = B^R \cdot A^R$.
- 2 $(A \cup B)^R = A^R \cup B^R$.

Which are the properties of the inverse of a language?

Definition

Given A and B languages over Σ ($A, B \subseteq \Sigma^*$).

- ❶ $(A \cdot B)^R = B^R \cdot A^R$.
- ❷ $(A \cup B)^R = A^R \cup B^R$.
- ❸ $(A \cap B)^R = A^R \cap B^R$.

Which are the properties of the inverse of a language?

Definition

Given A and B languages over Σ ($A, B \subseteq \Sigma^*$).

- ❶ $(A \cdot B)^R = B^R \cdot A^R$.
- ❷ $(A \cup B)^R = A^R \cup B^R$.
- ❸ $(A \cap B)^R = A^R \cap B^R$.
- ❹ $(A^R)^R = A$.

Which are the properties of the inverse of a language?

Definition

Given A and B languages over Σ ($A, B \subseteq \Sigma^*$).

- ❶ $(A \cdot B)^R = B^R \cdot A^R$.
- ❷ $(A \cup B)^R = A^R \cup B^R$.
- ❸ $(A \cap B)^R = A^R \cap B^R$.
- ❹ $(A^R)^R = A$.
- ❺ $(A^*)^R = (A^R)^*$.

Which are the properties of the inverse of a language?

Definition

Given A and B languages over Σ ($A, B \subseteq \Sigma^*$).

- ❶ $(A \cdot B)^R = B^R \cdot A^R$.
- ❷ $(A \cup B)^R = A^R \cup B^R$.
- ❸ $(A \cap B)^R = A^R \cap B^R$.
- ❹ $(A^R)^R = A$.
- ❺ $(A^*)^R = (A^R)^*$.
- ❻ $(A^+)^R = (A^R)^+$.

What are regular languages?

What are regular languages?

Regular languages from a given alphabet Σ are all those languages that can be built from the basic languages $\emptyset$, $\{\lambda\}$, $\{a\}$, $a \in \Sigma$, using union, concatenation and Kleene closure.

What are regular languages?

Regular languages from a given alphabet Σ are all those languages that can be built from the basic languages $\emptyset$, $\{\lambda\}$, $\{a\}$, $a \in \Sigma$, using union, concatenation and Kleene closure.

Given an alphabet Σ

What are regular languages?

Regular languages from a given alphabet Σ are all those languages that can be built from the basic languages $\emptyset$, $\{\lambda\}$, $\{a\}$, $a \in \Sigma$, using union, concatenation and Kleene closure.

Given an alphabet Σ

Definition

- 1 $\emptyset$, $\{\lambda\}$, $\{a\}$, for all $a \in \Sigma$, are regular languages over Σ . These are the so called basic regular languages.

What are regular languages?

Regular languages from a given alphabet Σ are all those languages that can be built from the basic languages $\emptyset$, $\{\lambda\}$, $\{a\}$, $a \in \Sigma$, using union, concatenation and Kleene closure.

Given an alphabet Σ

Definition

- 1 $\emptyset$, $\{\lambda\}$, $\{a\}$, for all $a \in \Sigma$, are regular languages over Σ . These are the so called basic regular languages.
- 2 If A and B are regular languages over Σ , then the following are also regular:

What are regular languages?

Regular languages from a given alphabet Σ are all those languages that can be built from the basic languages $\emptyset$, $\{\lambda\}$, $\{a\}$, $a \in \Sigma$, using union, concatenation and Kleene closure.

Given an alphabet Σ

Definition

- 1 $\emptyset$, $\{\lambda\}$, $\{a\}$, for all $a \in \Sigma$, are regular languages over Σ . These are the so called basic regular languages.
- 2 If A and B are regular languages over Σ , then the following are also regular:

$$A \cup B \quad (\text{union}),$$

What are regular languages?

Regular languages from a given alphabet Σ are all those languages that can be built from the basic languages $\emptyset$, $\{\lambda\}$, $\{a\}$, $a \in \Sigma$, using union, concatenation and Kleene closure.

Given an alphabet Σ

Definition

- 1 $\emptyset$, $\{\lambda\}$, $\{a\}$, for all $a \in \Sigma$, are regular languages over Σ . These are the so called basic regular languages.
- 2 If A and B are regular languages over Σ , then the following are also regular:

$$\begin{array}{ll} A \cup B & \text{(union),} \\ A \cdot B & \text{(concatenation),} \end{array}$$

What are regular languages?

Regular languages from a given alphabet Σ are all those languages that can be built from the basic languages $\emptyset$, $\{\lambda\}$, $\{a\}$, $a \in \Sigma$, using union, concatenation and Kleene closure.

Given an alphabet Σ

Definition

- 1 $\emptyset$, $\{\lambda\}$, $\{a\}$, for all $a \in \Sigma$, are regular languages over Σ . These are the so called basic regular languages.
- 2 If A and B are regular languages over Σ , then the following are also regular:

$A \cup B$ (union),
 $A \cdot B$ (concatenation),
 A^* (Kleene closure).

Σ and Σ^* are both regular languages over Σ . Union, concatenation and Kleene closure are known as regular operations.

Σ and Σ^* are both regular languages over Σ . Union, concatenation and Kleene closure are known as regular operations.

Example

Given $\Sigma = \{a, b\}$. The following ones are regular languages over Σ :

Σ and Σ^* are both regular languages over Σ . Union, concatenation and Kleene closure are known as regular operations.

Example

Given $\Sigma = \{a, b\}$. The following ones are regular languages over Σ :

- 1 The language A of all strings that have just one a :

$$A = \{b\}^* \cdot \{a\} \cdot \{b\}^*.$$

Σ and Σ^* are both regular languages over Σ . Union, concatenation and Kleene closure are known as regular operations.

Example

Given $\Sigma = \{a, b\}$. The following ones are regular languages over Σ :

- 1 The language A of all strings that have just one a :

$$A = \{b\}^* \cdot \{a\} \cdot \{b\}^*.$$

- 2 The language B of all strings that start with b :

$$B = \{b\} \cdot \{a, b\}^*.$$

Σ and Σ^* are both regular languages over Σ . Union, concatenation and Kleene closure are known as regular operations.

Example

Given $\Sigma = \{a, b\}$. The following ones are regular languages over Σ :

- ❶ The language A of all strings that have just one a :

$$A = \{b\}^* \cdot \{a\} \cdot \{b\}^*.$$

- ❷ The language B of all strings that start with b :

$$B = \{b\} \cdot \{a, b\}^*.$$

- ❸ The language C of all strings that contain the string ba :

$$C = \{a, b\}^* \cdot \{ba\} \cdot \{a, b\}^*.$$

Σ and Σ^* are both regular languages over Σ . Union, concatenation and Kleene closure are known as regular operations.

Example

Given $\Sigma = \{a, b\}$. The following ones are regular languages over Σ :

- ❶ The language A of all strings that have just one a :

$$A = \{b\}^* \cdot \{a\} \cdot \{b\}^*.$$

- ❷ The language B of all strings that start with b :

$$B = \{b\} \cdot \{a, b\}^*.$$

- ❸ The language C of all strings that contain the string ba :

$$C = \{a, b\}^* \cdot \{ba\} \cdot \{a, b\}^*.$$

- ❹ $(\{a\} \cup \{b\}^*) \cdot \{a\}$.

Σ and Σ^* are both regular languages over Σ . Union, concatenation and Kleene closure are known as regular operations.

Example

Given $\Sigma = \{a, b\}$. The following ones are regular languages over Σ :

- ❶ The language A of all strings that have just one a :

$$A = \{b\}^* \cdot \{a\} \cdot \{b\}^*.$$

- ❷ The language B of all strings that start with b :

$$B = \{b\} \cdot \{a, b\}^*.$$

- ❸ The language C of all strings that contain the string ba :

$$C = \{a, b\}^* \cdot \{ba\} \cdot \{a, b\}^*.$$

- ❹ $(\{a\} \cup \{b\}^*) \cdot \{a\}$.

- ❺ $[(\{a\}^* \cup \{b\}^*) \cdot \{b\}]^*$.

Agenda

- Power
- Kleene Closure
- Reverse
- Languages

1 Regular languages and expressions

- Regular Languages
- Regular Expressions

2 Regular expressions in Python

- Exercises in Python

Regular Languages

Regular languages from a given alphabet Σ are all those languages that can be built from the basic languages $\emptyset$, $\{\lambda\}$, $\{a\}$, $a \in \Sigma$, using union, concatenation and Kleene closure.

Regular Languages

Regular languages from a given alphabet Σ are all those languages that can be built from the basic languages $\emptyset$, $\{\lambda\}$, $\{a\}$, $a \in \Sigma$, using union, concatenation and Kleene closure.

Given an alphabet Σ

Definition

- 1 $\emptyset$, $\{\lambda\}$, $\{a\}$, for all $a \in \Sigma$, are regular languages over Σ . These are the so called basic regular languages.
- 2 If A and B are regular languages over Σ , then the following are also regular:

$A \cup B$ (union),
 $A \cdot B$ (concatenation),
 A^* (Kleene closure).

Regular Languages

Σ and Σ^* are both regular languages over Σ . Union, concatenation and Kleene closure are known as regular operations.

Example

Given $\Sigma = \{a, b\}$. The following ones are regular languages over Σ :

- ❶ The language A of all strings that have just one a :

$$A = \{b\}^* \cdot \{a\} \cdot \{b\}^*.$$

- ❷ The language B of all strings that start with b :

$$B = \{b\} \cdot \{a, b\}^*.$$

- ❸ The language C of all strings that contain the string ba :

$$C = \{a, b\}^* \cdot \{ba\} \cdot \{a, b\}^*.$$

- ❹ $(\{a\} \cup \{b\}^*) \cdot \{a\}$.

- ❺ $[(\{a\}^* \cup \{b\}^*) \cdot \{b\}]^*$.

- Power
- Kleene Closure
- Reverse
- Languages

1 Regular languages and expressions

- Regular Languages
- Regular Expressions

2 Regular expressions in Python

- Exercises in Python

Regular expression is a sequence of symbols and characters expressing a **Regular Language**, they are used as strings or patterns to be searched for within a longer piece of text.

Definition

① *Basic regular expressions:*

- $\emptyset$ is a regular expression that represents the language $\emptyset$
- λ is a regular expression that represents the language $\{\lambda\}$.
- a is a regular expression that represents the language $\{a\}$,
 $a \in \Sigma$.

② *If R and S are regular expressions over Σ , the following ones are also regular:*

$(R)(S)$
 $(R \cup S)$
 $(R)^*$

Example

Example

Given the alphabet $\Sigma = \{a, b, c\}$,

$$(a \cup b^*)a^*(bc)^*$$

is the regular expression that represents

$$(\{a\} \cup \{b\}^*) \cdot \{a\}^* \cdot \{bc\}^*$$

In programming, a regular expression is

- A pattern of text that consists of **literals** (ordinary characters) and special characters known as **metacharacters**.
- e.g. `a\w*`

We can also find

- Character classes (`[...]`)
- Negation of ranges (`[^...]`)
- Sets of regular expressions (`()`)
- Quantifiers
- Boundary matchers

What are literals?

- Are the simplest form of pattern matching in regular expression.
- They will simply succeed whenever that literal is found.
- If we apply the regular expression **fox** to search the phrase *The quick brown fox jumps over the lazy dog*, we will find one match.
- We can also obtain several results instead of just one, if we apply the regular expression `be` to the following phrase *To be, or not to be*.

What are metacharacters?

- A metacharacter is a character that has a special meaning during pattern processing.
- We use metacharacters in regular expressions to define the search criteria and any text manipulations.
- The simplest example of a metacharacter is the full stop.
- The full stop character matches any single character of any sort (apart from a newline).
- e.g. `.at` means: any letter, followed by the letter `a` followed by the letter `t`.

What are character classes?

- The character classes (also known as character sets) allow us to define a character that will match if any of the defined characters on the set is present.
- To define a character class, we should use the opening square bracket metacharacter `[`, then any accepted characters, and finally close with a closing square bracket `]`.
- e.g. `licen[cs]e`
- It is possible to also use the range of a character. This is done by leveraging the hyphen symbol (`-`) between two related characters.
- e.g. To match any lowercase letter we can use `[a-z]`.
- e.g. To match any single digit we can define the character set `[0-9]`.

How to work with several character classes?

- The character classes' ranges can be combined to be able to match a character against many ranges.
- We just need to put one range after the other.
- e.g. If we want to match any lowercase or uppercase alphanumeric character, we can use `[0-9a-zA-Z]`
- This can be alternatively written using the union mechanism:
`[0-9[a-zA-Z]]`.

What do we mean by negation of ranges?

- We can invert the meaning of a character set by placing a caret (^) symbol right after the opening square bracket metacharacter ([).
- If we have a character class such as `[0-9]` meaning any digit, the negated character class `[^0-9]` will match anything that is not a digit.

How to match against a set of regular expressions?

How to match against a set of regular expressions?

- This is accomplished by using the pipe symbol `|`.

How to match against a set of regular expressions?

- This is accomplished by using the pipe symbol `|`.
- It alternates between regular expressions.

How to match against a set of regular expressions?

- This is accomplished by using the pipe symbol `|`.
- It alternates between regular expressions.
- e.g. `yes|no`

How to match against a set of regular expressions?

- This is accomplished by using the pipe symbol `|`.
- It alternates between regular expressions.
- e.g. `yes|no`
- e.g. `yes|no|maybe`

An overview of regular expressions

What are quantifiers?

What are quantifiers?

- The mechanisms to define how a character, metacharacter, or character set can be repeated.

An overview of regular expressions

What are quantifiers?

- The mechanisms to define how a character, metacharacter, or character set can be repeated.

Which are the three basic quantifiers?

An overview of regular expressions

What are quantifiers?

- The mechanisms to define how a character, metacharacter, or character set can be repeated.

Which are the three basic quantifiers?

- ? 0 or 1 repetitions

An overview of regular expressions

What are quantifiers?

- The mechanisms to define how a character, metacharacter, or character set can be repeated.

Which are the three basic quantifiers?

- ? 0 or 1 repetitions
- * 0 or more times

An overview of regular expressions

What are quantifiers?

- The mechanisms to define how a character, metacharacter, or character set can be repeated.

Which are the three basic quantifiers?

- ? 0 or 1 repetitions
- * 0 or more times
- + 1 or more times

What are quantifiers?

- The mechanisms to define how a character, metacharacter, or character set can be repeated.

Which are the three basic quantifiers?

- `?` 0 or 1 repetitions
- `*` 0 or more times
- `+` 1 or more times
- `{n,m}` Between n and m times.

What are quantifiers?

- The mechanisms to define how a character, metacharacter, or character set can be repeated.

Which are the three basic quantifiers?

- `?` 0 or 1 repetitions
- `*` 0 or more times
- `+` 1 or more times
- `{n,m}` Between n and m times.
 - ▶ `{n}` Exactly n times.

What are quantifiers?

- The mechanisms to define how a character, metacharacter, or character set can be repeated.

Which are the three basic quantifiers?

- `?` 0 or 1 repetitions
- `*` 0 or more times
- `+` 1 or more times
- `{n,m}` Between n and m times.
 - ▶ `{n}` Exactly n times.
 - ▶ `{n, }` n or more times.

An overview of regular expressions

What are quantifiers?

- The mechanisms to define how a character, metacharacter, or character set can be repeated.

Which are the three basic quantifiers?

- `?` 0 or 1 repetitions
- `*` 0 or more times
- `+` 1 or more times
- `{n,m}` Between n and m times.
 - ▶ `{n}` Exactly n times.
 - ▶ `{n,}` n or more times.
 - ▶ `{,n}` At most n times.

An overview of regular expressions

What are quantifiers?

- The mechanisms to define how a character, metacharacter, or character set can be repeated.

Which are the three basic quantifiers?

- `?` 0 or 1 repetitions
- `*` 0 or more times
- `+` 1 or more times
- `{n,m}` Between *n* and *m* times.
 - ▶ `{n}` Exactly *n* times.
 - ▶ `{n, }` *n* or more times.
 - ▶ `{, n}` At most *n* times.
- e.g. `cars?` Singular or plural.

Example

How to match a telephone number that can be in the format 555-555-555, 555 555 555, or 555555555.

What are boundary matchers?

What are boundary matchers?

- The boundary matchers are a set of characters that will correspond to a particular position inside of the input.

What are boundary matchers?

- The boundary matchers are a set of characters that will correspond to a particular position inside of the input.
- `^` Matches at the beginning of a line

What are boundary matchers?

- The boundary matchers are a set of characters that will correspond to a particular position inside of the input.
- `^` Matches at the beginning of a line
- `$` Matches at the end of a line

What are boundary matchers?

- The boundary matchers are a set of characters that will correspond to a particular position inside of the input.
- `^` Matches at the beginning of a line
- `$` Matches at the end of a line
- `\b` Matches a word boundary

What are boundary matchers?

- The boundary matchers are a set of characters that will correspond to a particular position inside of the input.
- `^` Matches at the beginning of a line
- `$` Matches at the end of a line
- `\b` Matches a word boundary
- `\B` The opposite of `\b`

What are boundary matchers?

- The boundary matchers are a set of characters that will correspond to a particular position inside of the input.
- `^` Matches at the beginning of a line
- `$` Matches at the end of a line
- `\b` Matches a word boundary
- `\B` The opposite of `\b`
- `\A` Matches the beginning of the input

What are boundary matchers?

- The boundary matchers are a set of characters that will correspond to a particular position inside of the input.
- `^` Matches at the beginning of a line
- `$` Matches at the end of a line
- `\b` Matches a word boundary
- `\B` The opposite of `\b`
- `\A` Matches the beginning of the input
- `\Z` Matches the end of the input

Example

Write a regular expression that will match lines that start with `Name :` and make sure that after the name, there are only alphabetic characters or spaces until the end of the line.

Example

What is the difference between the regular expressions `hello` and `\bhello\b`?

How are regular expressions supported in Python?

- They are supported by the `re` module.
- We only need to import it to start using it.

How to start using them to match a pattern?

- We compile a pattern with `pattern = re.compile(r'foo')`
- We can try to match it against a string `pattern.match("foo bar")`

What are the building blocks for Python Regex?

- `RegexObject`
 - ▶ Also known as Pattern Object.
 - ▶ Represents a compiled regular expression.
- `MatchObject`
 - ▶ Represents the matched pattern.

What is a RegexObject?

- Before matching patterns we need to compile the regex.
- The compilation produces a reusable pattern object.
- This object provides all the operations that can be done (i.e. matching a pattern and finding all substrings that match a particular regex).
- e.g. `pattern = re.compile(r '<HTML>')`
- `pattern.match("<HTML>")`

What are two ways of matching a pattern?

- We can compile a pattern, which gives us a `RegexObject`.
- We can just use the module operations.
- If we compile a pattern we are able to reuse it.
- e.g. `pattern = re.compile(r'<HTML>')`
- `pattern.match("<HTML>")`
- e.g. `re.match(r'<HTML>', "<HTML>")`

How to search for string that match a pattern?

- In python we have two operations match and search.

How does match work?

- This method tries to match the compiled pattern only at the beginning of the string.
- If there is a match, then it returns a MatchObject.
- It has two optional parameters, pos and endpos.
- pos determines the position from where to search the pattern in the string.
- endpos determines the position until where the pattern is searched in the string.
- `pattern.match(string, pos, endpos)`

Example

Given `pattern = re.compile(r'^<HTML>')` what is the result of:

Example

Given `pattern = re.compile(r'^<HTML>')` what is the result of:

- `pattern.match("<HTML>")`
- `pattern.match(" <HTML>")`
- `pattern.match(" <HTML>"[2:])`

Example

Given `pattern = re.compile(r '<HTML>$')` what is the result of:

- `pattern.match("<HTML> ", 0, 6)`
- `pattern.match("<HTML> "[:6])`

How does search work?

- This operation would be like the match of many languages.
- It tries to match the pattern at any location of the string and not just at the beginning.
- If there is a match, then it returns a MatchObject.
- The pos and endpos parameters have the same meaning as that in the match operation.
- pos determines the position from where to search the pattern in the string.
- endpos determines the position until where the pattern is searched in the string.
- `pattern.match(string, pos, endpos)`

Example

Given `pattern = re.compile(r"world")` what is the result of:

- `pattern.match("hello world")`
- `pattern.match("hola mundo ")`

Example

Given `pattern = re.compile(r '^<HTML>', re.MULTILINE)`
what is the result of:

- `pattern.search("<HTML>")`
- `pattern.search(" <HTML>")`
- `pattern.search(" \n<HTML>")`
- `pattern.search(" \n<HTML>", 3)`
- `pattern.search("</div></body>\n<HTML>", 4)`
- `pattern.search(" \n<HTML>", 4)`

How does findall work?

- It returns a list with all the non-overlapping occurrences of a pattern and not the MatchObject like search and match do.
- e.g. `pattern = re.compile(r"\w+")`
- `pattern.findall("hello world")`
- `['hello', 'world']`

Example

- `pattern = re.compile(r"(\w+) (\w+) ")`
- `pattern.findall("Hello world hola mundo")`
- `[('Hello', 'world'), ('hola', 'mundo')]`

How does finditer work?

- Works essentially as as findall.
- It returns an iterator in which each element is a MatchObject.
- We can use the operations provided by this object.
- Useful when you need information for every match.

Example

- `pattern = re.compile(r"(\w+) (\w+) ")`
- `it = pattern.finditer("Hello world hola mundo")`
- `match = it.next()`
- `match.groups()`
- `('Hello', 'world')`
- `match.span()`
- `(0, 11)`

Which are some operations to modify strings?

Which are some operations to modify strings?

- `split(string, maxsplit=0)`: A string can be split based on the matches of the pattern.
- `sub(repl, string, count=0)`: Returns the resulting string after replacing the matched pattern in the original string with the replacement.

Example

- `pattern = re.compile(r"\W")`
- `pattern.split("Beautiful is better than ugly", 2)`
- `['Beautiful', 'is', 'better than ugly']`

Example

- `pattern = re.compile(r'[0-9]+')`
- `pattern.sub("-", "order0, order1 order13")`
- `'order- order- order-'`

What is a MatchObject?

What is a MatchObject?

- An object that represents the matched pattern.
- We will get one every time you execute one of these operations:
 - ▶ `match`
 - ▶ `search`
 - ▶ `finditer`
- Provides a set of operations for working with the captured groups.

Which are the main operations for a MatchObject?

Which are the main operations for a MatchObject?

- group
- groups
- groupdict
- start
- end
- span

What is the group operation?

What is the group operation?

- Gives the subgroups of the match.
- If invoked with no arguments or zero, returns the entire match.
- If one or more group identifiers are passed, the corresponding groups' matches will be returned.

Example

- `pattern = re.compile("(\w+) (\w+)")`
- `match = pattern.search("Hello world")`
- `match.group()` → `'Hello world'`
- `match.group(0)` → `'Hello world'`
- `match.group(1)` → `'Hello'`
- `match.group(2)` → `'world'`
- `match.group(0,2)` → `('Hello world', 'world')`

Example

- *Groups can be named.*
- *If the pattern has named groups, they can be accessed using the names or the index:*
- `pattern = re.compile(r"(?P<first>\w+) (?P<second>\w+) ")`
- `match = pattern.search("Hello world")`
- `match.group('first') → 'Hello'`
- `match.group(1) → 'Hello'`
- `match.group(0, 'first', 2) → ('Hello world', 'Hello', 'world')`

What is the groups operation?

What is the groups operation?

- It returns a tuple with all the subgroups in the match instead of giving one or some of the groups.

Example

- `pattern = re.compile("(\w+) (\w+)")`
- `match = pattern.search("Hello world")`
- `match.groups() → ('Hello', 'world')`

What is the groupdict operation?

What is the groupdict operation?

- It is used in the cases where named groups have been used.
- It will return a dictionary with all the groups that were found.
- `pattern = re.compile(r"(?P<first>\w+) (?P<second>\w+) ")`
- `match = pattern.search("Hello world")`
- `{'first': 'Hello', 'second': 'world'}`
- If there aren't named groups, then it returns an empty dictionary.

What about start, end and span operations?

What about start, end and span operations?

- start: returns the index where the pattern matched.
- end: returns the end of the substring matched by the group.
- span: gives a tuple with the values from start and end.

Agenda

- Power
- Kleene Closure
- Reverse
- Languages

1 Regular languages and expressions

- Regular Languages
- Regular Expressions

2 Regular expressions in Python

- Exercises in Python